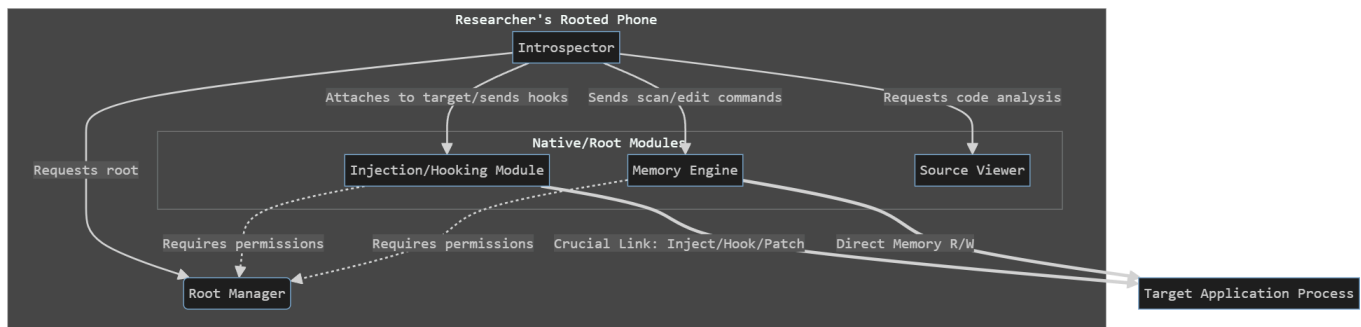


Introspector

Team: Constantin-Cosmin Alexa and Mihai-Adrian Tecliceanu

Architectural Diagram: Components and Connections



The Introspector application runs on a rooted researcher's phone, injecting code and manipulating memory in a target app's process. Think of it as several specialized tools working together.

Core Components

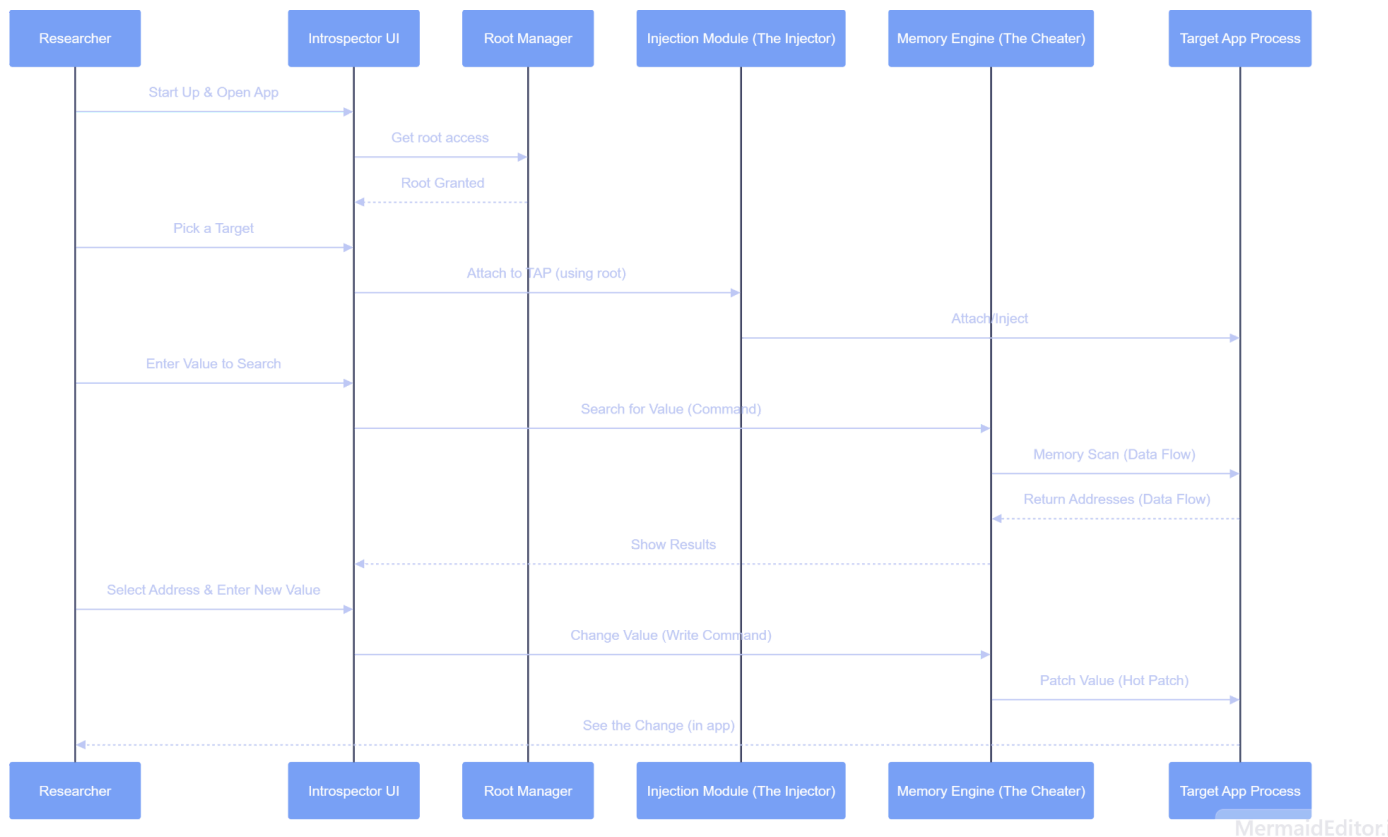
- **Introspector UI (The App):** The main Android interface. This is where you give commands, check the status, and see your results.
- **Root Manager:** Its job is to handle getting and keeping the necessary root access so we can mess with other processes.
- **Injection/Hooking Module (The Injector):** A native (C/C++) library that attaches to the target app's process using deep, root-level techniques (like `ptrace`). This is what enables hot patching and gives us that Frida-like capability.
- **Memory Engine (The Cheater):** A native module that reads and writes data directly into the target app's memory space, which powers the cheat engine feature.
- **Source Viewer (The Decompiler):** Handles pulling out, decompiling, and displaying the target app's code and resources, just like `jadx`.
- **Target Application Process:** The third-party Android app we're analyzing.

Connections (How it talks)

Source Component	Target Component	Description
Introspector UI ▾	Root Manager ▾	Requests root permissions when the app starts up.
Introspector UI ▾	The Injector ▾	Tells the Injector which target app to attach to.

Introspector UI ▾	The Cheater ▾	Sends memory scan/edit commands and gets the results back.
Introspector UI ▾	The Decompiler ▾	Asks to view the source code of a selected app.
The Injector ▾	Target App Process ▾	Crucial Link: Injects libraries or hooks into the target process for control and patching.
The Cheater ▾	Target App Process ▾	Reads and writes memory inside the running process (it needs The Injector to attach first).

Flow Diagram: Data and Command Flow (Cheat Engine)



Here's how the process works when a researcher uses the "cheat engine feature" to change a value in an app, like a score.

1. **Start Up & Get Root:** The researcher opens Introspector and gives it root access via the Root Manager.

2. **Pick a Target:** The researcher selects the app they want to analyze from the list in the UI.
3. **Attach:** The UI tells the Injector Module to attach to the target app's process using root permissions.
4. **Search for Value (Data Flow):** The researcher enters a value (like a score) to search for. The UI passes this command to the Memory Engine.
5. **Memory Scan:** The Memory Engine hunts through the target app's memory and identifies all addresses holding that value.
6. **Show Results:** The Engine sends the list of matching addresses back to the UI.
7. **Change Value (Command Flow):** The researcher selects an address and types in the new value (the 'cheat'). The UI sends this "Write" command to the Memory Engine.
8. **Patch:** The Memory Engine instantly writes the new value into the selected address in the target app's memory.
9. **See the Change:** The target app continues running, now reflecting the newly patched value.

Technical Planning

This is our technical roadmap, including all the necessary dev ops and coding steps to get Introspector built and released as open source.

Setup & Getting Started

1. Create the core Introspector project structure (Android app and native C/C++ modules).
2. Initialize the Git repository (since we're building this open source).
3. Set up CI/CD so builds and tests run automatically on every commit.
4. Configure code quality tools:
 - Define a unified code style for Kotlin conventions.
5. Finalize the detailed component design specs.

Core App & Injection

1. Build the main Introspector UI to show installed apps and the primary dashboard.
2. Implement the Root Manager to check for and request root privileges gracefully.
3. Develop the basic C/C++ Injector Module to attach to a target process ID (PID) and confirm we have control.

Memory Hacking (Cheat Engine-like)

1. Build the native Memory Engine functions to read and write memory in a remote process (like reading `/proc/<pid>/mem`).
2. Implement the memory scanning logic ("Exact Value Scan" and "Fuzzy Scan").
3. Integrate the memory scan UI so users can refine and change values easily.

Code Analysis & Patching

1. Research and integrate a library for the Source Viewer Module to analyze on-device

- DEX or bytecode (to get that `jadx`-like view).
2. Create the Source Viewer UI with proper syntax highlighting.
 3. Upgrade the Injection/Hooking Module to handle dynamic function hooking and loading custom scripts (like Frida).

Polish & Release

1. Run a final sweep with linters.
2. **Mandatory Code Operation:** Perform final comprehensive code reviews
3. **Open Source Deployment:** Publish the final, stable version of Introspector.